ENGINEERING PROJECT / RAG

October 2026 · Mihajlo Medic and Novo Sandara

DocPilot. RAG Chat System

Build a client-server chat system that answers questions about a documentation set you choose, using RAG. The server handles retrieval and generation; a .NET Framework 4.8 WPF client renders the answers as rich Markdown.

TECH STACK WPF / .NET Framework 4.8 · HTTP API · FAISS · Embeddings · LLM / SLM

3 stages	Any size	Optional
WEEKLY MILESTONES	EVALUATION DATASET	ALL BONUS FEATURES

01 / SYSTEM DESIGN

Architecture

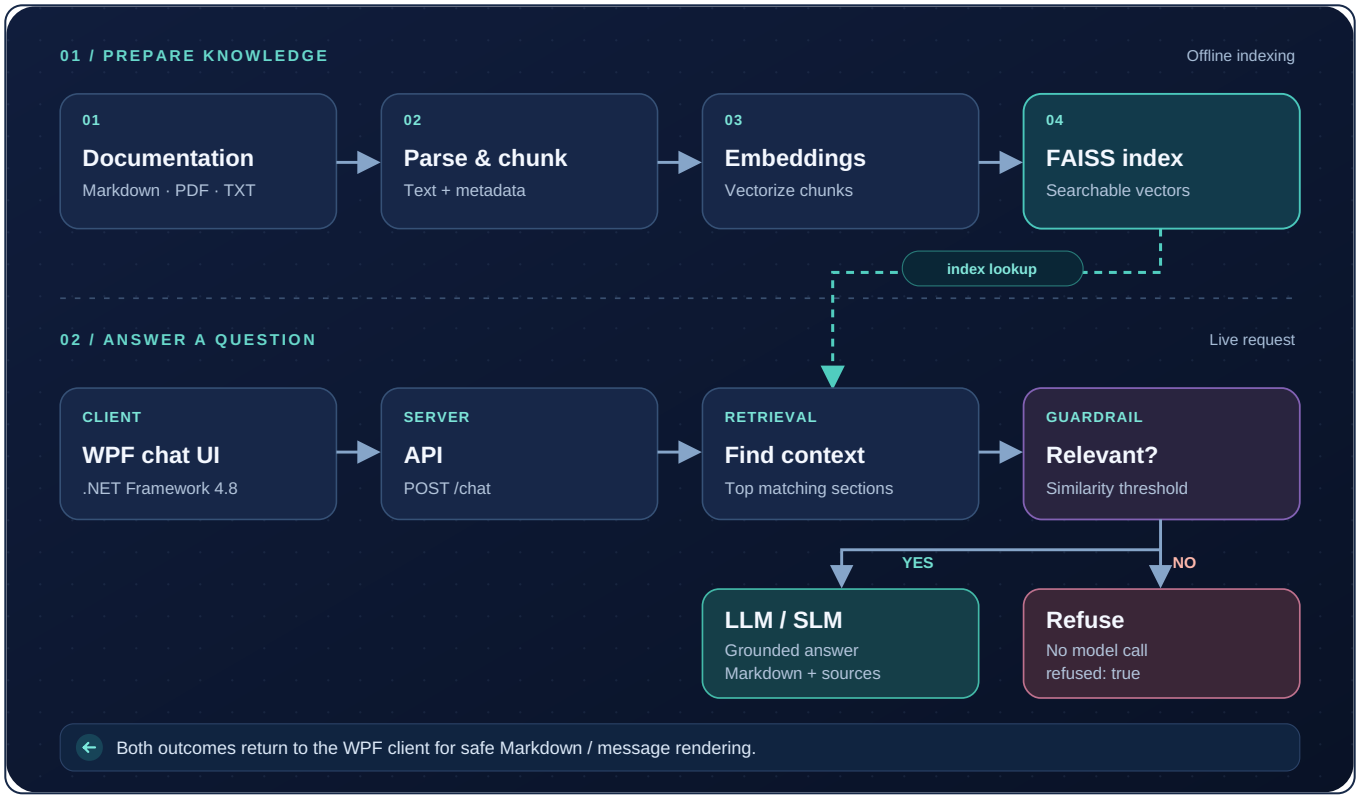


FIG. 01 End-to-end architecture · offline indexing + live question answering

The client sends a question and renders the Markdown answer it gets back. Everything else — guardrails, retrieval, and the model — lives on the server.

02 / KNOWLEDGE SOURCE

Documentation set

You choose the documentation. It must meet these requirements:

- Public, with a license that allows this use
- At least 50 pages, in Markdown or easy to convert to Markdown, including other formats such as PDF or TXT
- Contains tables, images, lists and code blocks, so the client has real content to render
- A topic you can judge answers on yourself

Good candidates: a folder of Microsoft's dotnet/docs, an open-source project's docs (Kubernetes, Home Assistant), or a product manual published in Markdown. Confirm your choice with your mentor by the end of day 2.

03 / BACKEND

Server

The server owns the whole RAG pipeline. Technology is your choice; Python is the natural fit because the RAG ecosystem lives there, but justify whatever you pick.

PART	REQUIREMENT	RECOMMENDATION
Ingestion	Parse docs, chunk them, keep metadata (file, heading path, URL, image links)	Chunk by heading, not by fixed character count
Embeddings	Your choice of model	A small open model, e.g. bge-small or nomic-embed-text
Vector store	Store and search chunk vectors	**FAISS** (IndexFlatIP with normalized vectors is enough at this size)
LLM / SLM	Your choice, local or a free API	Ollama with a 3–8B model runs locally at no cost
API	HTTP + JSON, documented	Choose a framework and document the API

API contract (minimum):

- `POST /chat` — request `{ question }`, response `{ answer (Markdown), sources[], refused }`
- `GET /health` — server and model status

Indexing can be a separate script; it doesn't need an endpoint. Answers are always returned as Markdown. Images can use publicly accessible HTTPS URLs from the chosen documentation; the server does not need to host them.

The mandatory implementation supports independent, single-turn questions. Conversation history and contextual follow-up questions are optional bonus features.

04 / FRONTEND

Client

The client is fixed: **.NET Framework 4.8 and WPF**. This is deliberate — we want to prove modern AI can sit behind a legacy desktop app.

Chat behavior:

- Message list with your questions and the assistant's answers, newest at the bottom
- Input box with a Send button; Enter sends, Shift+Enter adds a new line
- A loading indicator while the server works; the UI never freezes (async/await throughout)
- Clear error messages when the server is down or times out
- Each answer shows its sources as clickable links under the answer

Every answer must render correctly:

- Headings, paragraphs, bold and italic text
- Bulleted and numbered lists, including nested lists
- Tables
- Code blocks, ideally with syntax highlighting
- Images loaded from trusted HTTPS documentation URLs
- Links

Code quality: MVVM, `HttpClient` for the API calls, SDK-style project targeting `net48`.

05 / ENGINEERING RESEARCH

Markdown library research (mandatory)

Choosing the Markdown renderer is a research task, not a quick pick. Before writing client UI code, compare at least **three** options and justify your choice. Starting points to evaluate, not an approved list:

- **MdXaml** — Markdown to native WPF FlowDocument
- **Markdig.Wpf** — Markdig parser rendered to FlowDocument
- **Markdig to HTML in WebView2** — browser rendering inside WPF; needs the Edge WebView2 runtime on the machine
- Anything else you find that supports .NET Framework 4.8

Score each option on these criteria:

CRITERION	WHAT TO CHECK
.NET Framework 4.8 support	Installs and runs in a net48 WPF project
Tables	Borders, alignment, wide tables
Images	Loads images from trusted HTTPS documentation URLs
Lists	Nested bulleted and numbered lists
Code blocks	Monospace, ideally syntax highlighting
Styling	Fonts, colors, spacing match the app
Maintenance	Last release date, open issues, activity
License	Allowed for internal company use
Performance	Long answers render without UI lag

Deliverable: a one-page comparison table, your choice and why, and screenshots of one test Markdown file containing every element above, rendered by each candidate.

06 / SAFETY & RELIABILITY

Guardrails

Keep them simple; a few that work beat many that don't. Implement the core safeguards below. The items marked optional are additional improvements and are not required for completion:

01 **Grounded answers only.** The prompt tells the model to answer only from the retrieved sections.

02 **Refuse when retrieval is weak.** If the best similarity score is below a threshold, return "Not covered in the documentation" without calling the LLM, and set `refused: true`.

03 **Citations required.** Every answer lists the sections it used.

04 **Input checks.** Reject empty input and cap question length (for example 1,000 characters).

05 **Basic prompt-injection defense (optional).** Treat text inside documents and questions as data; test that instructions such as "ignore previous instructions" do not override the system's behavior.

06 **Safe rendering.** The client ignores raw HTML in answers and restricts image loading to trusted HTTPS sources.

07 **Additional server-side timeout controls (optional).** Consider limiting slow retrieval and model calls. The client must still handle timeouts and show readable errors, as required in the Client section.

07 / DELIVERY

Timeline and deliverables

Each week ends with a short demo to your mentor.

Week 1:

pick the docs, build ingestion, embeddings and FAISS search; `/chat` returns the top 3 sections without an LLM. Complete the Markdown library research.

Week 2:

add the LLM and guardrails; build the WPF chat client with the chosen Markdown library.

Week 3:

evaluate, fix the worst failures, polish, and present.

Evaluation: Create a dataset with any number of test questions you choose. Include both questions the documentation can answer and questions it cannot answer. Record the expected sources for answerable questions. Report how often the correct source appears in the top 3 retrieved sections, how often unanswerable questions are correctly refused, and answer quality on a simple 1–3 scale.

Done means:

- [ ] Markdown library research document
- [ ] Evaluation results
- [ ] Demo

08 / OPTIONAL EXTENSIONS

Bonus points

The following features are completely optional and should only be attempted after all mandatory requirements are implemented, tested, and demonstrated successfully. Implementing none of them does not affect successful completion of the project.

Streaming answers
— tokens appear as they're generated (server-sent events), rendered smoothly in WPF

Follow-up questions
— conversation history so "and how do I configure it?" works

Hybrid search
— combine keyword search (BM25) with vectors

Reranking
— a small reranker model to reorder the retrieved sections

Jump to source
— clicking a citation shows the exact section

Feedback buttons
— thumbs up and down, logged on the server

Docker
— the server starts with one command

Dark mode
in the client

Comparing models
— run the evaluation against two LLMs or two embedding models and report the difference